

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
THE UNIVERSITY OF TEXAS AT ARLINGTON**

**SYSTEM REQUIREMENTS SPECIFICATION
CSE 4316: SENIOR DESIGN I
SPRING 2026**



**FINAL ROUND
AI INTERVIEWER**

**UJJWAL SOOD
SAAD KHAIRULLAH
ABEL SANCHEZ
VAMSHI VAVILLA
CHRISTIAN HERNANDEZ
DEEP AVAIYA**

REVISION HISTORY

Revision	Date	Author(s)	Description
0.1	03.24.2026	AS, DA	Imported course SRS template and prepared initial document structure
0.2	03.31.2026	Team	Drafted product concept, product description, and major requirement sections
0.3	04.12.2026	Team	Expanded requirement sections and revised remaining document content
1.0	04.17.2026	Team	Completed full SRS draft and prepared submission version

CONTENTS

1	Product Concept	10
1.1	Purpose and Use	10
1.2	Intended Audience	10
2	Product Description	11
2.1	Features & Functions	11
2.2	External Inputs & Outputs	11
2.3	Product Interfaces	12
3	Customer Requirements	14
3.1	Account Access and Interview Setup	14
3.1.1	Description	14
3.1.2	Source	14
3.1.3	Constraints	14
3.1.4	Standards	14
3.1.5	Priority	14
3.2	Personalized Resume-Aware Questioning	14
3.2.1	Description	14
3.2.2	Source	14
3.2.3	Constraints	14
3.2.4	Standards	14
3.2.5	Priority	14
3.3	Technical Coding Interview Workflow	15
3.3.1	Description	15
3.3.2	Source	15
3.3.3	Constraints	15
3.3.4	Standards	15
3.3.5	Priority	15
3.4	Behavioral and Company-Specific Interviewing	15
3.4.1	Description	15
3.4.2	Source	15
3.4.3	Constraints	15
3.4.4	Standards	15
3.4.5	Priority	15
3.5	Feedback Dashboard and Session History	15
3.5.1	Description	15
3.5.2	Source	16
3.5.3	Constraints	16
3.5.4	Standards	16
3.5.5	Priority	16
3.6	Speech-Enabled Interview Mode	16
3.6.1	Description	16
3.6.2	Source	16
3.6.3	Constraints	16
3.6.4	Standards	16
3.6.5	Priority	16

4	Packaging Requirements	17
4.1	Containerized Deployment Package	17
4.1.1	Description	17
4.1.2	Source	17
4.1.3	Constraints	17
4.1.4	Standards	17
4.1.5	Priority	17
4.2	Browser-Based Delivery	17
4.2.1	Description	17
4.2.2	Source	17
4.2.3	Constraints	17
4.2.4	Standards	17
4.2.5	Priority	17
4.3	Local Assets and Model Packaging	17
4.3.1	Description	17
4.3.2	Source	18
4.3.3	Constraints	18
4.3.4	Standards	18
4.3.5	Priority	18
4.4	Administrator Setup Documentation	18
4.4.1	Description	18
4.4.2	Source	18
4.4.3	Constraints	18
4.4.4	Standards	18
4.4.5	Priority	18
5	Performance Requirements	19
5.1	Session Startup Time	19
5.1.1	Description	19
5.1.2	Source	19
5.1.3	Constraints	19
5.1.4	Standards	19
5.1.5	Priority	19
5.2	Interactive Text Response Latency	19
5.2.1	Description	19
5.2.2	Source	19
5.2.3	Constraints	19
5.2.4	Standards	19
5.2.5	Priority	19
5.3	Session Persistence and Recovery	19
5.3.1	Description	20
5.3.2	Source	20
5.3.3	Constraints	20
5.3.4	Standards	20
5.3.5	Priority	20
5.4	Prototype Demo Capacity	20
5.4.1	Description	20
5.4.2	Source	20

5.4.3	Constraints	20
5.4.4	Standards	20
5.4.5	Priority	20
5.5	Small-Group Concurrent Sessions	20
5.5.1	Description	20
5.5.2	Source	20
5.5.3	Constraints	21
5.5.4	Standards	21
5.5.5	Priority	21
5.6	Spoken Response Round-Trip Time	21
5.6.1	Description	21
5.6.2	Source	21
5.6.3	Constraints	21
5.6.4	Standards	21
5.6.5	Priority	21
6	Safety Requirements	22
6.1	Controlled Code Execution Environment	22
6.1.1	Description	22
6.1.2	Source	22
6.1.3	Constraints	22
6.1.4	Standards	22
6.1.5	Priority	22
6.2	Graceful Failure and Fallback Handling	22
6.2.1	Description	22
6.2.2	Source	22
6.2.3	Constraints	22
6.2.4	Standards	22
6.2.5	Priority	22
6.3	Upload Validation and Resource Limits	22
6.3.1	Description	22
6.3.2	Source	23
6.3.3	Constraints	23
6.3.4	Standards	23
6.3.5	Priority	23
6.4	Advisory Use and Output Transparency	23
6.4.1	Description	23
6.4.2	Source	23
6.4.3	Constraints	23
6.4.4	Standards	23
6.4.5	Priority	23
7	Security Requirements	24
7.1	Authenticated Access and Session Control	24
7.1.1	Description	24
7.1.2	Source	24
7.1.3	Constraints	24
7.1.4	Standards	24

7.1.5	Priority	24
7.2	Protected Transport and Credential Storage	24
7.2.1	Description	24
7.2.2	Source	24
7.2.3	Constraints	24
7.2.4	Standards	24
7.2.5	Priority	24
7.3	Data Protection and Retention Control	24
7.3.1	Description	24
7.3.2	Source	25
7.3.3	Constraints	25
7.3.4	Standards	25
7.3.5	Priority	25
7.4	Audit Logging and Security-Relevant Events	25
7.4.1	Description	25
7.4.2	Source	25
7.4.3	Constraints	25
7.4.4	Standards	25
7.4.5	Priority	25
7.5	Dependency and Configuration Hygiene	25
7.5.1	Description	25
7.5.2	Source	25
7.5.3	Constraints	25
7.5.4	Standards	26
7.5.5	Priority	26
8	Maintenance & Support Requirements	27
8.1	Version-Controlled Source and Deployment Artifacts	27
8.1.1	Description	27
8.1.2	Source	27
8.1.3	Constraints	27
8.1.4	Standards	27
8.1.5	Priority	27
8.2	Operational Runbook and Troubleshooting Guide	27
8.2.1	Description	27
8.2.2	Source	27
8.2.3	Constraints	27
8.2.4	Standards	27
8.2.5	Priority	27
8.3	Prompt and Model Configuration Management	27
8.3.1	Description	27
8.3.2	Source	28
8.3.3	Constraints	28
8.3.4	Standards	28
8.3.5	Priority	28
8.4	Backup, Restore, and Data Export Support	28
8.4.1	Description	28
8.4.2	Source	28

8.4.3	Constraints	28
8.4.4	Standards	28
8.4.5	Priority	28
9	Other Requirements	29
9.1	Modular Service Architecture	29
9.1.1	Description	29
9.1.2	Source	29
9.1.3	Constraints	29
9.1.4	Standards	29
9.1.5	Priority	29
9.2	Portable Local Development Environment	29
9.2.1	Description	29
9.2.2	Source	29
9.2.3	Constraints	29
9.2.4	Standards	29
9.2.5	Priority	29
9.3	Testability and Regression Support	29
9.3.1	Description	29
9.3.2	Source	30
9.3.3	Constraints	30
9.3.4	Standards	30
9.3.5	Priority	30
9.4	Pluggable Fine-Tuned Model Packs	30
9.4.1	Description	30
9.4.2	Source	30
9.4.3	Constraints	30
9.4.4	Standards	30
9.4.5	Priority	30
10	Future Items	31
10.1	Speech-Enabled Interview Mode	31
10.1.1	Description	31
10.1.2	Source	31
10.1.3	Constraints	31
10.1.4	Standards	31
10.1.5	Priority	31
10.2	Small-Group Concurrent Sessions	31
10.2.1	Description	31
10.2.2	Source	31
10.2.3	Constraints	31
10.2.4	Standards	31
10.2.5	Priority	31
10.3	Spoken Response Round-Trip Time	31
10.3.1	Description	31
10.3.2	Source	31
10.3.3	Constraints	32
10.3.4	Standards	32

10.3.5 Priority	32
10.4 Pluggable Fine-Tuned Model Packs	32
10.4.1 Description	32
10.4.2 Source	32
10.4.3 Constraints	32
10.4.4 Standards	32
10.4.5 Priority	32

LIST OF FIGURES

1	Conceptual landing page for Final Round	10
2	High-level system overview for Final Round	12
3	Login interface used to access the platform	13
4	Recent session history interface	13

1 PRODUCT CONCEPT

Final Round is being built as a single platform for software engineering interview practice. Our goal is to give candidates one place where they can work through technical, resume-based, and behavioral interview rounds instead of jumping across several different tools.

1.1 PURPOSE AND USE

The product is meant to simulate a realistic software engineering interview through a web browser. A user signs in, selects a target role or company-style interview, uploads a resume, and starts a session. From there, the system moves through coding prompts, resume follow-up questions, and behavioral questions in one continuous flow.

We are building Final Round as an active practice tool rather than a passive study resource. Instead of only showing sample questions, the system reacts to what the user writes, keeps the interview moving, and gives structured feedback at the end of the session. The current version focuses on a text-first interview flow because it is easier for us to test, evaluate, and demonstrate reliably on local infrastructure. Voice interaction is planned for a later stage once the text pipeline is stable.

1.2 INTENDED AUDIENCE

The main audience for Final Round is undergraduate computer science students and early-career candidates preparing for internships, new graduate roles, and entry-level software engineering interviews. These users need something affordable, available on demand, and realistic enough to show where they are weak before they sit in an actual interview.

Secondary users include faculty reviewers, senior design staff, and university career support groups that may want to use the platform as a demo system or as a scalable interview-practice tool. The product is not intended to replace a live interviewer or make hiring decisions. Its purpose is to help users practice, reflect on their performance, and improve across repeated sessions.

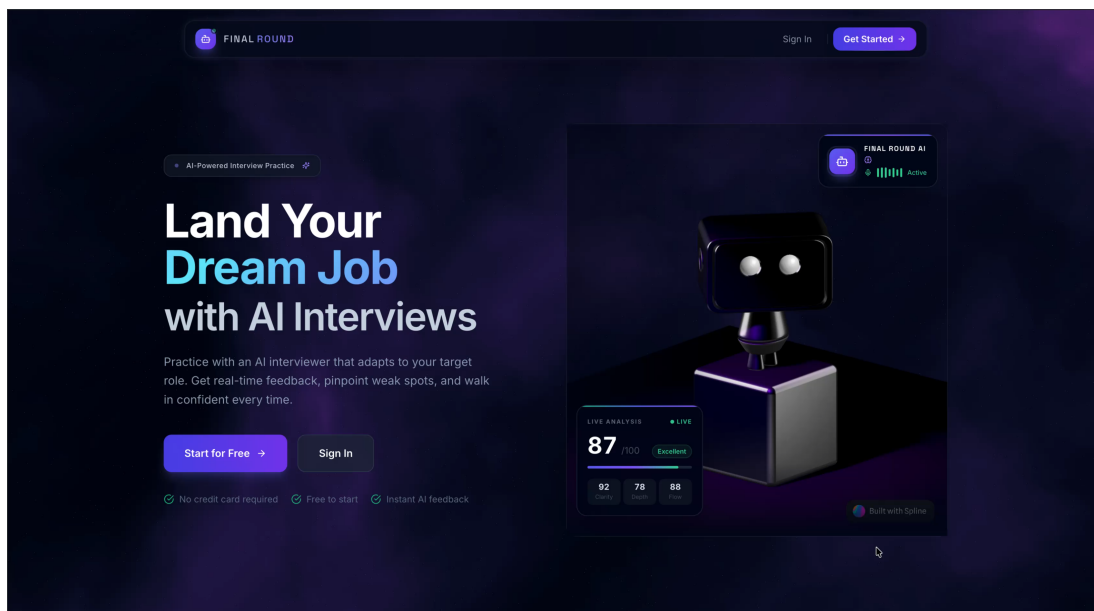


Figure 1: Conceptual landing page for Final Round

2 PRODUCT DESCRIPTION

Final Round combines the main parts of software engineering interview preparation into one platform. The system is designed to take a user from setup, to interview, to feedback without forcing them to switch between unrelated tools.

2.1 FEATURES & FUNCTIONS

In the current scope, Final Round is expected to provide the following main capabilities:

- user sign-in and session management through a browser-based interface;
- target role or company-style selection before the interview starts;
- local resume upload and parsing so the interviewer can ask grounded follow-up questions;
- a structured interview flow with coding, resume, and behavioral sections;
- feedback that summarizes strengths, weak points, and practical next steps; and
- a recent-session view so users can revisit earlier practice attempts.

The platform is intended for interview practice. It does not make hiring decisions, replace employer screening systems, or guarantee a user will perform well in a real interview. When code execution is supported, it will run inside a controlled environment. When full execution is not available, the system will still evaluate the logic, structure, and completeness of the submitted answer.

2.2 EXTERNAL INPUTS & OUTPUTS

The major inputs and outputs for the system are shown in Table 2. These cover the main information that enters the platform from the user and the main results the platform returns.

Table 2: Critical external inputs and outputs

Name	Description	Use
Account credentials	Email and password or similar sign-in data used to authenticate a user session.	Grants access to interview history, saved sessions, and protected routes.
Role and company selection	Candidate-selected target context such as software engineer, internship, or a company-style interview profile.	Guides prompt selection, difficulty, and evaluation tone.
Resume document	Uploaded PDF or document containing education, projects, skills, and experience.	Parsed locally so the interviewer can ask personalized follow-up questions.
User interview responses	Free-text answers, coding responses, and later voice responses when enabled.	Drives interview branching, transcript generation, and evaluation.
Interview prompts and follow-ups	Questions generated by the orchestration and model layers.	Delivered to the candidate in sequence to simulate the interview.
Feedback report	Structured strengths, weaknesses, suggested improvements, and session summary.	Returned to the user after a session and stored for later review.
Session and audit logs	System events, timestamps, and service-level status information.	Supports debugging, maintenance, and limited administrative review.

2.3 PRODUCT INTERFACES

The high-level relationship between the main subsystems is shown in Figure 2. The browser-based user interface sends user actions to the interview orchestration layer, which coordinates model calls, code analysis, and persistent storage.

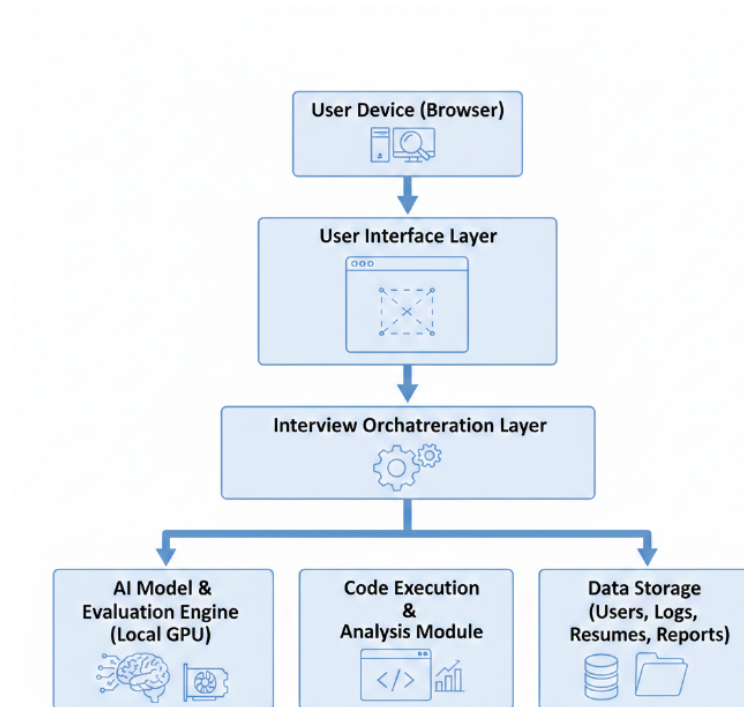


Figure 2: High-level system overview for Final Round

The current prototype also includes visible interface components that matter directly to end users. The login screen shown in Figure 3 reflects the current minimal design approach used before an interview session begins.

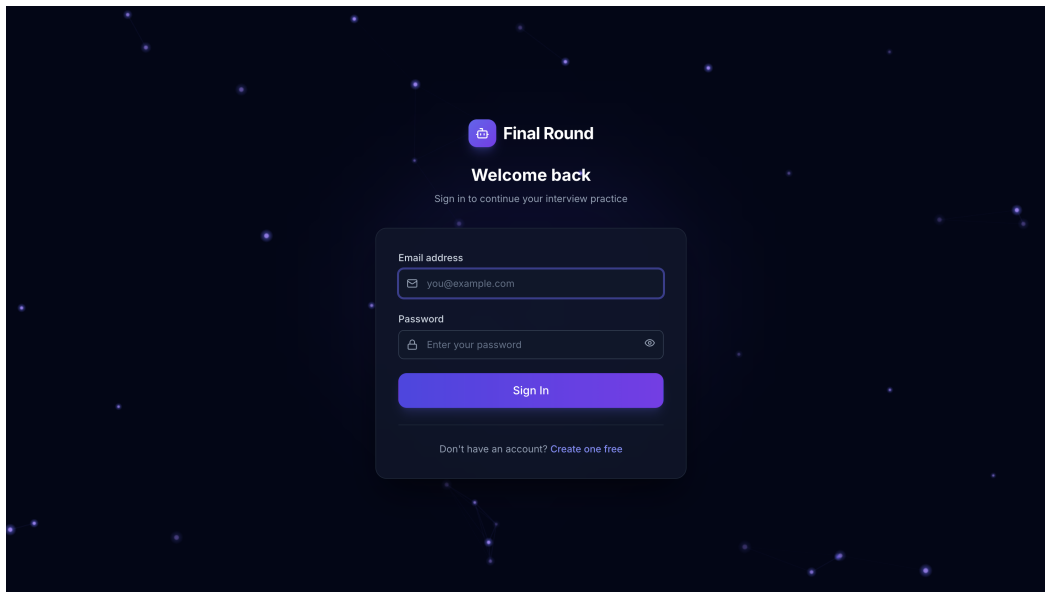


Figure 3: Login interface used to access the platform

After a user completes practice sessions, the recent-session interface shown in Figure 4 allows them to reopen earlier work and review progress across attempts.

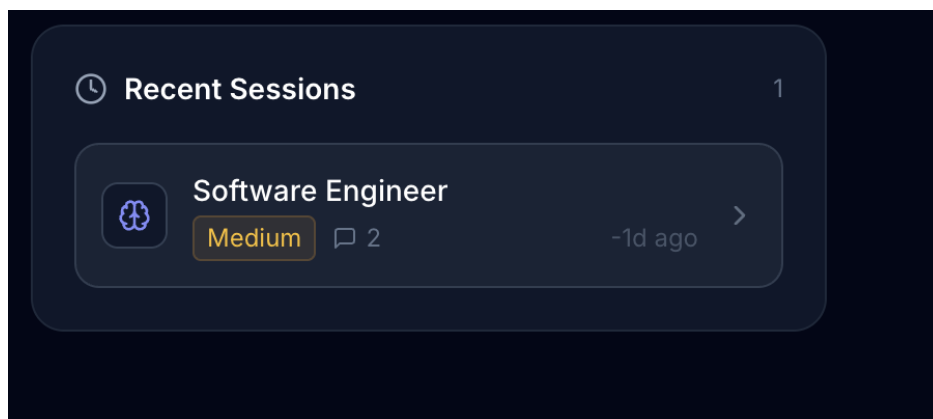


Figure 4: Recent session history interface

3 CUSTOMER REQUIREMENTS

The following requirements describe the main features a user or reviewer should be able to see directly in Final Round.

3.1 ACCOUNT ACCESS AND INTERVIEW SETUP

3.1.1 DESCRIPTION

The system shall provide a browser-based sign-in flow and an interview setup flow that lets a user access the product, choose a target interview context, and begin a practice session without manual developer assistance. At minimum, the setup flow shall support account access, role selection, company or interview-style selection, and session start from a standard desktop browser.

3.1.2 SOURCE

Project charter, sponsor expectations, and Sprint 2 through Sprint 3 UI work.

3.1.3 CONSTRAINTS

The setup flow must remain simple enough for class demonstrations and repeat use by students. It must run on institution-approved hardware and common desktop browsers without requiring a native application install.

3.1.4 STANDARDS

The user-facing workflow should follow common web usability conventions and stay consistent with WCAG 2.2 accessibility guidance for navigation, form labels, and visible focus states [1].

3.1.5 PRIORITY

Critical

3.2 PERSONALIZED RESUME-AWARE QUESTIONING

3.2.1 DESCRIPTION

The system shall allow a user to upload a resume and shall use locally extracted resume content to generate interview questions that reference the candidate's projects, skills, coursework, and prior experience. Follow-up questions shall stay grounded in information actually present in the uploaded resume rather than invented details.

3.2.2 SOURCE

Project charter methodology and value proposition.

3.2.3 CONSTRAINTS

Resume parsing quality may vary depending on document formatting. The system must therefore validate allowed file types, enforce upload limits, and fail gracefully when a resume cannot be parsed with enough confidence.

3.2.4 STANDARDS

Input validation and file handling should follow secure application guidance such as OWASP ASVS practices for file upload, validation, and access control [2].

3.2.5 PRIORITY

Critical

3.3 TECHNICAL CODING INTERVIEW WORKFLOW

3.3.1 DESCRIPTION

The system shall present coding questions inside the interview workflow and shall accept a typed solution from the user. The platform shall evaluate the submission either by controlled execution in a sandboxed environment or, when execution is disabled, by analyzing logic, structure, and completeness. The candidate must be able to submit work without leaving the platform.

3.3.2 SOURCE

Project charter problem statement and Sprint 2 backend goals.

3.3.3 CONSTRAINTS

Runtime evaluation must not compromise host safety or platform stability. Available compute resources in the senior design environment may limit the range of languages, test depth, or execution time supported by the prototype.

3.3.4 STANDARDS

The implementation should align with secure software development practices and controlled execution guidance informed by NIST SSDF and internal sandbox policies [3].

3.3.5 PRIORITY

Critical

3.4 BEHAVIORAL AND COMPANY-SPECIFIC INTERVIEWING

3.4.1 DESCRIPTION

The system shall support behavioral and company-context interview prompts in addition to coding questions. When a user selects a company profile or target interview style, the questioning should reflect that selection through tone, emphasis, and follow-up style while remaining realistic and professional.

3.4.2 SOURCE

Project charter problem statement, methodology, and development milestones.

3.4.3 CONSTRAINTS

The realism of company-specific behavior depends on prompt design, locally available reference data, and model quality. The system must not imply that it has access to confidential internal interview banks.

3.4.4 STANDARDS

Prompt sets and evaluation rubrics shall be versioned in the team repository so that interview behavior can be reviewed and updated in a controlled way.

3.4.5 PRIORITY

High

3.5 FEEDBACK DASHBOARD AND SESSION HISTORY

3.5.1 DESCRIPTION

At the end of a session, the system shall present a feedback view that identifies observed strengths, weak areas, and practical next steps. The system shall also store recent session summaries so a returning user can review earlier attempts and keep practicing over time.

3.5.2 SOURCE

Project charter value proposition and Sprint 3 work related to session history and feedback layout.

3.5.3 CONSTRAINTS

Feedback quality must stay specific enough to be useful. Generic praise without evidence should be avoided. Stored session history must respect retention, privacy, and available storage limits.

3.5.4 STANDARDS

User-visible reports should preserve readability and interface accessibility in line with WCAG 2.2 guidance [1].

3.5.5 PRIORITY

High

3.6 SPEECH-ENABLED INTERVIEW MODE

3.6.1 DESCRIPTION

A future release should allow the user to speak with the interviewer through local speech-to-text and text-to-speech components. The goal of this requirement is to make the interview feel closer to a live conversation while preserving the same session structure used by the text-first prototype.

3.6.2 SOURCE

Sprint 3 speech-to-text work and Sprint 4 plan for local TTS and STT integration.

3.6.3 CONSTRAINTS

This feature depends on microphone access, local audio model performance, and acceptable latency on available hardware. It is outside the minimum demo scope for the current prototype.

3.6.4 STANDARDS

Browser audio permissions and stored audio handling must follow standard browser security constraints and local privacy controls.

3.6.5 PRIORITY

Future

4 PACKAGING REQUIREMENTS

The packaging requirements for Final Round focus on how the system is delivered, started, and handed off to another team member, reviewer, or instructor.

4.1 CONTAINERIZED DEPLOYMENT PACKAGE

4.1.1 DESCRIPTION

The delivered prototype shall be packaged as a containerized application stack with a frontend service, backend service, and supporting local components that can be started together through a documented orchestration file such as `docker-compose.yml`. This packaging must allow a maintainer to reproduce the demo environment with minimal manual setup.

4.1.2 SOURCE

Sprint 4 plan, technical architecture decisions, and deployment needs for class demonstrations.

4.1.3 CONSTRAINTS

The package must remain compatible with institution-approved lab machines. GPU acceleration may not be present on every development machine, so deployment should separate required configuration from optional performance tuning.

4.1.4 STANDARDS

Containers should follow standard OCI-compatible image practices, and environment-specific secrets must remain external to the image build.

4.1.5 PRIORITY

Critical

4.2 BROWSER-BASED DELIVERY

4.2.1 DESCRIPTION

The product shall be delivered to end users as a web application that runs in a modern desktop browser. No thick-client installation should be required for normal use of the prototype.

4.2.2 SOURCE

Project charter system overview and sponsor expectations for easy demo access.

4.2.3 CONSTRAINTS

The supported environment for the prototype is a desktop browser on lab or student hardware. Mobile optimization is not required in the first release.

4.2.4 STANDARDS

The delivered interface should rely on broadly supported web standards, including HTML, CSS, and JavaScript behavior compatible with current Chromium- and Firefox-based browsers.

4.2.5 PRIORITY

Critical

4.3 LOCAL ASSETS AND MODEL PACKAGING

4.3.1 DESCRIPTION

Wherever feasible in the prototype, model files, prompts, and stored user artifacts shall remain on institution-approved infrastructure rather than depending on third-party hosted inference services. Con-

figuration files and local model paths shall be clearly documented for maintainers.

4.3.2 SOURCE

Project charter assumptions, constraints, and system overview.

4.3.3 CONSTRAINTS

Licensing, disk capacity, and GPU memory may limit which local models can be bundled directly with the delivery package. Large model files may need to be documented as a post-install step instead of committed to the source repository.

4.3.4 STANDARDS

Software assets and dependencies must be used according to their stated license terms. Sensitive configuration values must not be hard-coded into distributable artifacts.

4.3.5 PRIORITY

High

4.4 ADMINISTRATOR SETUP DOCUMENTATION

4.4.1 DESCRIPTION

The delivered package shall include setup and operating documentation that explains prerequisites, environment variables, startup commands, shutdown procedures, and basic troubleshooting steps. The documentation shall be sufficient for another team member or instructor to start the system without relying on undocumented setup knowledge.

4.4.2 SOURCE

Sponsor expectations and maintenance needs for project handoff.

4.4.3 CONSTRAINTS

Documentation quality must be maintained alongside code changes. If deployment steps change, the written instructions must be updated before the corresponding build is treated as complete.

4.4.4 STANDARDS

The setup guide should be version-controlled with the source code and written in a way that supports reproducible builds and repeatable demonstrations.

4.4.5 PRIORITY

High

5 PERFORMANCE REQUIREMENTS

The following requirements define the expected performance of Final Round under normal prototype conditions. The targets are intended to keep the platform usable during practice sessions and class demonstrations rather than to represent production-scale service levels.

5.1 SESSION STARTUP TIME

5.1.1 DESCRIPTION

After an authenticated user begins an interview, the system shall present the first interview prompt within 10 seconds under normal local demo conditions. This timing includes retrieving the configured interview context and initializing session state.

5.1.2 SOURCE

Sponsor expectations for a responsive demo and general usability needs.

5.1.3 CONSTRAINTS

Startup time depends on local service readiness, model warm-up behavior, and whether resume parsing is required before the first question can be shown.

5.1.4 STANDARDS

No external standard is mandated, but measured startup behavior should be tracked during sprint testing and regression checks.

5.1.5 PRIORITY

High

5.2 INTERACTIVE TEXT RESPONSE LATENCY

5.2.1 DESCRIPTION

For the text-first prototype, the system shall return the next question, follow-up, or short evaluation response within 8 seconds for a typical interaction on the supported local environment. If processing exceeds this threshold, the interface shall show a visible loading or progress state so the user knows the system is still working.

5.2.2 SOURCE

Project usability goals and the need to preserve interview flow.

5.2.3 CONSTRAINTS

Latency is affected by model size, local GPU availability, prompt complexity, and whether code analysis is triggered in the same step.

5.2.4 STANDARDS

Performance measurements should be logged consistently during internal testing so the team can compare model and prompt changes between sprints.

5.2.5 PRIORITY

Critical

5.3 SESSION PERSISTENCE AND RECOVERY

5.3.1 DESCRIPTION

The platform shall save interview progress frequently enough that an unexpected browser refresh or non-destructive service restart does not lose more than one user turn. At minimum, completed prompts, user answers, and generated feedback summaries shall be written to persistent storage during the session.

5.3.2 SOURCE

Need for reliable iterative practice and session history features.

5.3.3 CONSTRAINTS

The persistence design must balance responsiveness, storage cost, and implementation effort. Overly frequent writes can create unnecessary overhead on limited local hardware.

5.3.4 STANDARDS

Persistence logic should be covered by regression tests for create, read, and restore behavior.

5.3.5 PRIORITY

High

5.4 PROTOTYPE DEMO CAPACITY

5.4.1 DESCRIPTION

The system shall support at least one active interview session and one additional browser session for observation or administrative validation on the same deployment without crashing or corrupting saved state. This requirement reflects the minimum environment expected during senior design demonstrations.

5.4.2 SOURCE

Class demonstration requirements and local deployment architecture.

5.4.3 CONSTRAINTS

The prototype is not yet sized for broad public traffic. Available memory, model footprint, and code execution overhead constrain the number of simultaneously active sessions that can be supported at this stage.

5.4.4 STANDARDS

Internal load checks should be documented with the same test environment used during demos so results remain interpretable.

5.4.5 PRIORITY

Moderate

5.5 SMALL-GROUP CONCURRENT SESSIONS

5.5.1 DESCRIPTION

A future release should support a small number of concurrent interview sessions for use by a lab section, club workshop, or classroom demonstration without requiring a full architectural rewrite.

5.5.2 SOURCE

Project charter note on future expansion to multi-user concurrency.

5.5.3 CONSTRAINTS

This capability depends on available compute, queueing strategy, and stable model-serving behavior. It is not part of the minimum acceptable prototype.

5.5.4 STANDARDS

Future scalability work should preserve session isolation and not weaken the security controls defined elsewhere in this document.

5.5.5 PRIORITY

Future

5.6 SPOKEN RESPONSE ROUND-TRIP TIME

5.6.1 DESCRIPTION

When local speech features are enabled in a future release, the combined speech-to-text, model response, and text-to-speech pipeline should complete quickly enough to preserve conversational flow. The target for a normal response is 5 seconds or less on supported hardware.

5.6.2 SOURCE

Sprint 4 voice-interaction planning.

5.6.3 CONSTRAINTS

Audio latency depends heavily on model size, device performance, and microphone quality. It is outside the minimum scope of the current text-first prototype.

5.6.4 STANDARDS

Audio performance should be benchmarked with the same local hardware profile used for deployment decisions.

5.6.5 PRIORITY

Future

6 SAFETY REQUIREMENTS

Even though Final Round is a software platform, it still needs safety-oriented controls because it processes uploaded files, accepts code-like input, and runs on shared institutional hardware.

6.1 CONTROLLED CODE EXECUTION ENVIRONMENT

6.1.1 DESCRIPTION

Any feature that executes or evaluates user-submitted code shall do so in a controlled environment with explicit resource limits, restricted permissions, and no unnecessary access to host services. If a safe execution path is unavailable, the platform shall fall back to static or logical analysis rather than running untrusted code directly on the host.

6.1.2 SOURCE

Project charter system overview and the need to evaluate coding responses safely.

6.1.3 CONSTRAINTS

Strict isolation can reduce execution fidelity and may limit supported languages or available libraries. The implementation must prefer safety over feature breadth.

6.1.4 STANDARDS

Safe execution practices should align with secure software development guidance such as NIST SSDF and internal lab computing rules [3].

6.1.5 PRIORITY

Critical

6.2 GRACEFUL FAILURE AND FALLBACK HANDLING

6.2.1 DESCRIPTION

If a dependent service fails, such as resume parsing, model inference, or local speech components, the system shall fail in a controlled and user-readable way. Error states shall not expose raw stack traces to normal users, and the platform shall prefer a degraded but usable mode whenever feasible.

6.2.2 SOURCE

Sprint 4 planning for fallback behavior and practical demo needs.

6.2.3 CONSTRAINTS

Graceful degradation requires extra state handling and testing effort. Not every component can offer a meaningful fallback, but the user should still receive a clear message about what happened.

6.2.4 STANDARDS

Operational failures should be logged for maintainers while user-facing messages remain concise and non-sensitive.

6.2.5 PRIORITY

High

6.3 UPLOAD VALIDATION AND RESOURCE LIMITS

6.3.1 DESCRIPTION

The system shall validate uploaded resumes and other user-supplied artifacts before processing them. Allowed file types, file sizes, and parsing paths shall be restricted so malformed or oversized files do not

destabilize the application.

6.3.2 SOURCE

Need to safely process user resumes and preserve shared lab resources.

6.3.3 CONSTRAINTS

Tight limits can reject legitimate but unusually formatted documents. Thresholds must therefore be conservative enough to protect the system while still supporting normal student resumes.

6.3.4 STANDARDS

Input validation and file handling shall be consistent with secure upload guidance reflected in OWASP ASVS [2].

6.3.5 PRIORITY

High

6.4 ADVISORY USE AND OUTPUT TRANSPARENCY

6.4.1 DESCRIPTION

The product shall present itself as an interview-practice and feedback tool, not as a hiring authority. User-facing messaging shall avoid claims that the generated score is final, official, or guaranteed to reflect employer outcomes.

6.4.2 SOURCE

Project scope, sponsor expectations, and ethical use concerns.

6.4.3 CONSTRAINTS

The interface must remain clear without overloading the user with warnings. Short, plain-language notices are preferred.

6.4.4 STANDARDS

No single formal standard governs this feature, but the requirement is intended to support responsible AI use and reduce misinterpretation of system output.

6.4.5 PRIORITY

High

7 SECURITY REQUIREMENTS

Because Final Round stores resumes, interview transcripts, and evaluation reports, the platform must protect user data and prevent misuse of both the application and the local infrastructure it runs on.

7.1 AUTHENTICATED ACCESS AND SESSION CONTROL

7.1.1 DESCRIPTION

Protected routes in the platform shall require user authentication. Session tokens or equivalent credentials shall expire after a defined period of inactivity, and a user shall not be able to access another user's saved interviews or reports through simple URL changes or client-side manipulation.

7.1.2 SOURCE

Basic platform security expectations and user account design.

7.1.3 CONSTRAINTS

The prototype should remain easy to demo, but convenience cannot come from disabling access control on stored user data.

7.1.4 STANDARDS

Authentication and access control behavior should align with OWASP ASVS guidance for session management and authorization [2].

7.1.5 PRIORITY

Critical

7.2 PROTECTED TRANSPORT AND CREDENTIAL STORAGE

7.2.1 DESCRIPTION

Credentials shall never be stored in plaintext. Passwords or equivalent secrets shall be protected using a modern password hashing approach, and network transport for deployed instances shall use encrypted channels whenever the system is accessed over a network.

7.2.2 SOURCE

Standard software security practice and sponsor expectations.

7.2.3 CONSTRAINTS

Local development environments may use self-signed certificates or isolated lab networking during setup, but the deployed demonstration environment must still handle secrets responsibly.

7.2.4 STANDARDS

Encrypted transport should follow TLS 1.3 guidance [4], and password hashing should follow modern memory-hard recommendations such as Argon2 usage guidance [5].

7.2.5 PRIORITY

Critical

7.3 DATA PROTECTION AND RETENTION CONTROL

7.3.1 DESCRIPTION

Resumes, transcripts, code submissions, and feedback reports shall be stored only as long as needed for project use, debugging, or user review. Administrative access to stored data shall be limited, and

deletion or reset procedures shall be documented so the team can remove stale data from demonstration environments.

7.3.2 SOURCE

Project charter constraints related to privacy and institutional infrastructure.

7.3.3 CONSTRAINTS

The project is not building a full enterprise retention platform, so retention controls must be practical for a student team to operate.

7.3.4 STANDARDS

Storage and access policies should follow least-privilege principles and secure software development practices described by NIST SSDF [3].

7.3.5 PRIORITY

High

7.4 AUDIT LOGGING AND SECURITY-RELEVANT EVENTS

7.4.1 DESCRIPTION

The platform shall record security-relevant events such as login attempts, failed parsing events, service start or stop actions, and controlled execution failures. Logs should be useful for troubleshooting and abuse detection without storing unnecessary sensitive content.

7.4.2 SOURCE

Operational support needs and QA responsibilities assigned within the team.

7.4.3 CONSTRAINTS

Overly verbose logging can expose user data or consume limited storage. Log content must therefore be scoped carefully.

7.4.4 STANDARDS

Logging practices should support traceability and review in line with secure development and operational monitoring guidance [3].

7.4.5 PRIORITY

High

7.5 DEPENDENCY AND CONFIGURATION HYGIENE

7.5.1 DESCRIPTION

The system shall track its third-party dependencies and separate code from environment-specific configuration. Secrets, API keys, and private paths shall not be committed directly into the source repository or baked into distributable container images.

7.5.2 SOURCE

Deployment packaging needs and secure team workflow expectations.

7.5.3 CONSTRAINTS

Student development speed can tempt hard-coded shortcuts. This requirement exists to prevent that habit from reaching the delivered build.

7.5.4 STANDARDS

Dependency and secret-management hygiene should follow secure software supply-chain practices and repository-level review controls.

7.5.5 PRIORITY

High

8 MAINTENANCE & SUPPORT REQUIREMENTS

Maintenance and support requirements define what must be available after delivery so the product can be started, repaired, and extended by current team members or future maintainers.

8.1 VERSION-CONTROLLED SOURCE AND DEPLOYMENT ARTIFACTS

8.1.1 DESCRIPTION

All source code, infrastructure files, prompt templates, and deployment scripts required to rebuild the prototype shall be maintained in version control. The delivered repository shall identify the baseline configuration used for the submitted demo build.

8.1.2 SOURCE

Team workflow and long-term maintainability needs.

8.1.3 CONSTRAINTS

Large local model weights may need to remain outside the repository, but their required versions and retrieval steps must still be documented clearly.

8.1.4 STANDARDS

Repository history shall be treated as the system of record for code, configuration, and prompt changes that affect released behavior.

8.1.5 PRIORITY

Critical

8.2 OPERATIONAL RUNBOOK AND TROUBLESHOOTING GUIDE

8.2.1 DESCRIPTION

The project shall include a maintainer-facing runbook that explains how to start services, inspect logs, verify model availability, reset the database if necessary, and recover from common demo-time failures.

8.2.2 SOURCE

Need for reliable support during demonstrations and future handoff.

8.2.3 CONSTRAINTS

The guide must stay synchronized with actual deployment scripts. Outdated troubleshooting notes create false confidence and slow recovery.

8.2.4 STANDARDS

Runbook steps should be tested on a clean environment at least once before the related release is treated as stable.

8.2.5 PRIORITY

High

8.3 PROMPT AND MODEL CONFIGURATION MANAGEMENT

8.3.1 DESCRIPTION

Interview prompts, scoring rubrics, model names, and relevant threshold values shall be stored in a way that allows maintainers to update them without rewriting unrelated application code. Any change that affects interview behavior must be traceable to a documented configuration or repository revision.

8.3.2 SOURCE

AI interview logic workstream and expected future tuning.

8.3.3 CONSTRAINTS

Over-configuring the system can make it harder to understand. Only parameters that genuinely affect maintainability should be externalized.

8.3.4 STANDARDS

Behavior-changing configuration must be reviewable and versioned with the same discipline applied to source code.

8.3.5 PRIORITY

High

8.4 BACKUP, RESTORE, AND DATA EXPORT SUPPORT

8.4.1 DESCRIPTION

The platform shall provide a practical way to back up and restore essential stored artifacts, including user records, session summaries, and evaluation outputs. Where full backup automation is not implemented, manual export steps shall be documented for maintainers.

8.4.2 SOURCE

Need to preserve work between sprints and avoid accidental data loss.

8.4.3 CONSTRAINTS

The prototype does not require enterprise-grade disaster recovery, but it should avoid needless data loss in a student development environment.

8.4.4 STANDARDS

Backup and restore procedures should be tested against a representative local deployment before final delivery.

8.4.5 PRIORITY

Moderate

9 OTHER REQUIREMENTS

The following requirements do not fit cleanly into the earlier categories, but they still matter to the completeness and long-term stability of Final Round.

9.1 MODULAR SERVICE ARCHITECTURE

9.1.1 DESCRIPTION

The application shall separate major responsibilities such as user interface delivery, interview orchestration, model interaction, code analysis, and persistence into clear modules or services. This requirement exists so one part of the system can be improved without forcing unrelated rewrites across the entire stack.

9.1.2 SOURCE

Project charter system overview and technical lead architecture goals.

9.1.3 CONSTRAINTS

Over-separation can create unnecessary complexity for a student project. The architecture should be modular, but not fragmented beyond what the team can realistically maintain.

9.1.4 STANDARDS

Module boundaries and interfaces should be documented in the design artifacts produced after this SRS.

9.1.5 PRIORITY

High

9.2 PORTABLE LOCAL DEVELOPMENT ENVIRONMENT

9.2.1 DESCRIPTION

A team member shall be able to run the project on a supported local development machine without reproducing the entire lab setup by hand. Differences between lab deployment and personal development environments should be expressed through configuration rather than source-code forks.

9.2.2 SOURCE

Team collaboration needs and Docker-based deployment direction.

9.2.3 CONSTRAINTS

Not every developer machine will support the same model sizes or acceleration features. A reduced local mode may be necessary for some contributors.

9.2.4 STANDARDS

Environment differences should be documented through version-controlled configuration files and reproducible setup instructions.

9.2.5 PRIORITY

Moderate

9.3 TESTABILITY AND REGRESSION SUPPORT

9.3.1 DESCRIPTION

Core workflows such as sign-in, session creation, resume ingestion, interview progression, and feedback generation shall be structured so they can be verified repeatedly during QA. At minimum, the system shall support a repeatable test checklist and bug-tracking workflow for sprint reviews.

9.3.2 SOURCE

Team QA responsibilities and the need to avoid regressions as features are added.

9.3.3 CONSTRAINTS

Full automated end-to-end coverage may not be realistic for every feature in the prototype. The team should therefore combine targeted automation with disciplined manual regression checks.

9.3.4 STANDARDS

Testing artifacts should be maintained in the same project-management workflow used to track implementation status.

9.3.5 PRIORITY

High

9.4 PLUGGABLE FINE-TUNED MODEL PACKS

9.4.1 DESCRIPTION

A future release should allow the team to swap between supported local models, such as Gemma-, Qwen-, or similar interview-tuned variants, without redesigning the user interface or storage layer. This requirement is intended to support later experiments in interview realism and response quality.

9.4.2 SOURCE

Sprint 4 plan for model fine-tuning and local AI experimentation.

9.4.3 CONSTRAINTS

Model interchangeability depends on consistent prompt contracts, hardware limits, and careful evaluation. It is not required for the minimum acceptable prototype.

9.4.4 STANDARDS

Future model integrations should preserve the security, logging, and evaluation controls already established elsewhere in this specification.

9.4.5 PRIORITY

Future

10 FUTURE ITEMS

The following items are intentionally deferred beyond the minimum prototype scope. We considered them during planning, but current time, hardware, and integration limits make them better targets for a later release.

10.1 SPEECH-ENABLED INTERVIEW MODE

10.1.1 DESCRIPTION

A later release should allow users to conduct the interview through local speech-to-text and text-to-speech components so the interaction feels closer to a live interviewer conversation.

10.1.2 SOURCE

Sprint 3 speech-to-text work and Sprint 4 TTS and STT planning.

10.1.3 CONSTRAINTS

This feature depends on acceptable audio latency, stable microphone handling, and local model performance on supported hardware.

10.1.4 STANDARDS

Future audio handling must preserve local privacy controls and browser permission rules.

10.1.5 PRIORITY

Future

10.2 SMALL-GROUP CONCURRENT SESSIONS

10.2.1 DESCRIPTION

A future version should support a small number of simultaneous interview sessions for workshops, class demonstrations, or lab usage without requiring a major redesign of the core platform.

10.2.2 SOURCE

Project charter note on future multi-user concurrency.

10.2.3 CONSTRAINTS

This requirement depends on model-serving stability, queueing strategy, and available compute capacity.

10.2.4 STANDARDS

Future scaling work must preserve session isolation and security controls.

10.2.5 PRIORITY

Future

10.3 SPOKEN RESPONSE ROUND-TRIP TIME

10.3.1 DESCRIPTION

When speech mode is introduced, the end-to-end audio loop should respond fast enough to preserve normal conversational rhythm, with a target of roughly 5 seconds or less on supported hardware.

10.3.2 SOURCE

Voice interaction performance goals from Sprint 4 planning.

10.3.3 CONSTRAINTS

Audio performance is sensitive to model size, hardware class, and microphone quality.

10.3.4 STANDARDS

Benchmarking must use a documented hardware profile so results are reproducible.

10.3.5 PRIORITY

Future

10.4 PLUGGABLE FINE-TUNED MODEL PACKS

10.4.1 DESCRIPTION

A future release should make it practical to compare and swap locally hosted interview-tuned models without reworking the rest of the application stack.

10.4.2 SOURCE

Sprint 4 model fine-tuning scope and long-term architecture direction.

10.4.3 CONSTRAINTS

Model interchangeability requires shared interfaces, disciplined prompt templates, and enough hardware to test several candidate models.

10.4.4 STANDARDS

Any future model additions must continue to meet the security and logging controls defined in this document.

10.4.5 PRIORITY

Future

REFERENCES

- [1] World Wide Web Consortium, “Web Content Accessibility Guidelines (WCAG) 2.2,” W3C Recommendation, 2023.
- [2] OWASP Foundation, “Application Security Verification Standard,” security verification guidance, 2023.
- [3] National Institute of Standards and Technology, “Secure Software Development Framework (SSDF),” SP 800-218, 2022.
- [4] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446, 2018.
- [5] A. Biryukov, D. Dinu, and D. Khovratovich, “Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications,” RFC 9106, 2021.